

Trick no 1 (Prefix Minimum Trick)

What is the Prefix Minimum Trick?

Suppose we need to choose two indices l and r , where

$$l < r,$$

and maximize an expression of the following form:

$$F(l, r) = X(r) - Y(l).$$

A brute-force solution checks every possible pair (l, r) , which requires

$$O(n^2)$$

time.

However, while processing a fixed right endpoint r , the value $X(r)$ is constant. Therefore,

$$\max_{l < r} (X(r) - Y(l)) = X(r) - \min_{l < r} Y(l).$$

Thus, instead of checking every previous index l , we only need to remember the minimum value of $Y(l)$ among all previously processed indices.

This stored value is called the **prefix minimum**.

Basic Example

Suppose we want to maximize

$$a_r - a_l,$$

where

$$l < r.$$

For a fixed r , the value a_r is constant. Therefore, to maximize the expression, we need to choose the smallest previous value of a_l .

Hence,

$$\text{best value for } r = a_r - \min(a_1, a_2, \dots, a_{r-1}).$$

Consider the following array:

$$a = [7, 2, 6, 1, 8].$$

While scanning the array from left to right:

r	a_r	Minimum before r	Best difference
2	2	7	-5
3	6	2	4
4	1	2	-1
5	8	1	7

The maximum answer is

$$8 - 1 = 7.$$

Instead of checking every possible pair, we continuously maintain the smallest previous value.

The Main Algebraic Pattern

Whenever a pair expression can be rearranged into

$$F(l, r) = P(r) - Q(l),$$

where $l < r$, we can write

$$\max F(l, r) = P(r) - \min_{l < r} Q(l).$$

Similarly, if the expression is

$$F(l, r) = P(r) + Q(l),$$

then

$$\max F(l, r) = P(r) + \max_{l < r} Q(l).$$

The main objective is to separate the expression into two independent parts:

a part depending only on r

and

a part depending only on l .

Example Involving Indices

Suppose we need to maximize

$$F(l, r) = a_r - a_l + r - l.$$

At first, the expression appears to depend on both l and r . We can rearrange it as follows:

$$F(l, r) = a_r + r - a_l - l,$$

or

$$F(l, r) = (a_r + r) - (a_l + l).$$

For a fixed r , the first part

$$a_r + r$$

is constant.

Therefore, to maximize the expression, we need the minimum previous value of

$$a_l + l.$$

Thus, the candidate answer for every r is

$$(a_r + r) - \min_{l < r} (a_l + l).$$

This reduces the time complexity from

$$O(n^2)$$

to

$$O(n).$$

```
long long mn = a[0];
long long ans = LLONG_MIN;

for (int r = 1; r < n; r++) {
    ans = max(ans, a[r] + r - mn);
    mn = min(mn, a[r] + r);
}
```

The order of the operations is important:

1. Calculate the answer using only previously processed indices.
2. Update the prefix minimum using the current index.

This guarantees that

$$l < r$$

instead of allowing

$$l \leq r.$$

When Should We Think About This Trick?

We should think about the prefix minimum or prefix maximum trick when the following conditions appear.

1. We Need to Choose Two Indices

Usually, the problem asks us to choose two indices satisfying

$$l < r.$$

The brute-force solution tries every possible pair.

2. The Constraints Do Not Allow $O(n^2)$

For example, if

$$n \leq 2 \cdot 10^5,$$

then an $O(n^2)$ solution will be too slow.

This usually indicates that an $O(n)$ or $O(n \log n)$ solution is required.

3. One Index Can Be Used as the Current Position

We can scan the right endpoint r from left to right and choose the best previous left endpoint l .

For every fixed r , we should ask:

What information about all previous indices l is enough to calculate the best answer?

Sometimes, we only need:

- the minimum previous value,
- the maximum previous value,
- the minimum transformed value,
- the maximum transformed value.

4. The Expression Can Be Separated Algebraically

We should try to transform

$$F(l, r)$$

into

$$P(r) + Q(l)$$

or

$$P(r) - Q(l).$$

For example,

$$a_r - a_l + r - l$$

can be transformed into

$$(a_r + r) - (a_l + l).$$

This algebraic separation is the biggest indication that the prefix minimum or prefix maximum trick can be used.

Useful Recognition Table

Expression to maximize	Value to maintain
$P(r) - Q(l)$	Minimum previous $Q(l)$
$P(r) + Q(l)$	Maximum previous $Q(l)$
$Q(l) - P(r)$	Maximum previous $Q(l)$
$-P(r) - Q(l)$	Minimum previous $Q(l)$

The general rule is:

- If the previous-index expression is being subtracted, maintain its minimum.
- If the previous-index expression is being added, maintain its maximum.

Example with Absolute Values

Suppose we need to calculate

$$\max_{i < j} (|a_i - a_j| + |i - j|).$$

Since

$$i < j,$$

we have

$$|i - j| = j - i.$$

However, the expression

$$|a_i - a_j|$$

has two possible cases.

Case 1

Suppose

$$|a_i - a_j| = a_j - a_i.$$

Then

$$a_j - a_i + j - i$$

can be rearranged as

$$(a_j + j) - (a_i + i).$$

Therefore, we maintain the minimum previous value of

$$a_i + i.$$

Case 2

Suppose

$$|a_i - a_j| = a_i - a_j.$$

Then

$$a_i - a_j + j - i$$

can be rearranged as

$$(j - a_j) - (i - a_i).$$

Therefore, we maintain the minimum previous value of

$$i - a_i.$$

This is another common pattern: absolute-value expressions can produce multiple cases, and every case may require a separate prefix minimum or prefix maximum.

Maintaining Multiple Prefix Minimums

Sometimes, valid previous indices belong to different categories, such as:

- odd indices,
- even indices,
- values having different remainders modulo k ,
- different characters,
- different colors,
- different dynamic programming states.

In such cases, we can maintain one prefix minimum for each category.

For example, for odd and even indices:

```
const long long INF = 4e18;

long long mn[2] = {INF, INF};

for (int r = 0; r < n; r++) {
    int parity = r % 2;

    // Calculate candidates using the required category.

    long long transformedValue = a[r] + r;
    mn[parity] = min(mn[parity], transformedValue);
}
```

Therefore, the trick is not limited to storing only one minimum. We may store separate minimum or maximum values for different valid groups.

General Mental Process

Whenever we see a pair optimization problem, we can follow these steps.

Step 1: Fix the Right Endpoint

Assume that r is fixed.

Then ask:

Which previous index $l < r$ gives the optimal answer?

Step 2: Rearrange the Expression

Try to write

$$F(l, r) = P(r) + Q(l)$$

or

$$F(l, r) = P(r) - Q(l).$$

Step 3: Determine the Required Previous Information

For a fixed r , determine whether we need:

- the smallest previous transformed value,
- the largest previous transformed value,
- one minimum or maximum for each category.

Step 4: Scan from Left to Right

Use the stored prefix information to calculate the best candidate for the current r .

Step 5: Update After Querying

The general structure is:

```
for (int r = 0; r < n; r++) {  
    // Use only previous indices.  
    calculate_answer();  
  
    // Add the current index for future positions.  
    update_prefix_information();  
}
```

Calculating the answer before updating ensures that the current position is not incorrectly used as both l and r .

When Does This Trick Not Work Directly?

The prefix minimum trick does not work directly when the contribution of l still depends on r in a complicated manner.

For example,

$$F(l, r) = a_l \cdot a_r.$$

The optimal previous a_l depends on whether a_r is positive or negative.

This may still be handled by maintaining both the minimum and maximum previous values, but one prefix minimum alone is not sufficient.

Consider another example:

$$F(l, r) = (a_l - a_r)^2.$$

Expanding the expression gives

$$F(l, r) = a_l^2 - 2a_l a_r + a_r^2.$$

The term

$$-2a_l a_r$$

depends on both l and r . Therefore, the expression cannot be directly separated into

$$P(r) + Q(l).$$

In such situations, more advanced optimization techniques may be required.

One-Sentence Intuition

Prefix minimum optimization means that while scanning the right endpoint, we store the best transformed value among all possible previous left endpoints, so that we never need to check those left endpoints individually again.

Practice Problems

1. Codeforces 2140C:

<https://codeforces.com/problemset/problem/2140/C>

2. Codeforces 2145C:

<https://codeforces.com/problemset/problem/2145/C>